

CSCI3180 Principles of Programming Languages

Ryan Chan

May 6, 2026

Abstract

This is a note for **CSCI3180 Principles of Programming Languages**.

Contents are adapted from the lecture notes of CSCI3180, prepared by [Lauren Pick](#), as well as some online resources.

This note is intended solely as a study aid. While I have done my best to ensure the accuracy of the content, I do not take responsibility for any errors or inaccuracies that may be present. Please use the material thoughtfully and at your own discretion.

If you believe any part of this content infringes on copyright, feel free to contact me, and I will address it promptly.

Mistakes might be found. So please feel free to point out any mistakes.

Contents

1	Foundations	2
1.1	Syntax and Semantics	2
1.2	Formal Syntax Rules	2
1.3	Abstract syntax trees (ASTs)	5
1.4	Beyond BNF and Meta-variables	6
2	Formal Proof Systems	9
2.1	Proof Systems	9
2.2	From Grammars to Proof System	12
2.3	Predicate Logic	12
3	Semantics Basics	14
3.1	Introduction to Semantics	14
3.2	SIMPL	14
3.3	Small-step Operational Semantics	15
3.4	Evaluation Order and Determinism	17
3.5	Big-step Operational Semantics	19
3.6	Syntactic Sugar	22

Chapter 1

Foundations

1.1 Syntax and Semantics

Consider an arbitrary programming language $\mathcal{L}$ (a set of programs). We have:

- **Syntax**: describes which things are programs in $\mathcal{L}$
- **Semantics**: describes what happens when we run a program in $\mathcal{L}$

So, given the syntax for a programming language $\mathcal{L}$, with a set of values $\mathcal{V}$, we define

$$\text{Interpreter}_{\mathcal{L}} : \mathcal{L} \rightarrow \mathcal{V}$$

Here the *interpreter* takes a program in $\mathcal{L}$ and runs it to produce a value in $\mathcal{V}$. For example,

$$\text{Interpreter}_{\text{Python}}(\text{sorted}([1, 4, 3, 2])) = [1, 2, 3, 4]$$

This can be seen as one way of defining the semantics of the program.

A compiler translates from one language to another. For a program p in $\mathcal{L}_1$ and $\text{Compiler} : \mathcal{L}_1 \rightarrow \mathcal{L}_2$ to be correct, we want

$$\text{Interpreter}_{\mathcal{L}_1}(p) = \text{Interpreter}_{\mathcal{L}_2}(\text{Compiler}(p))$$

otherwise something is incorrect.

Notice that interpreters and compilers, in **theory**, are mathematical objects, but not in reality. They have implementations, and while we can treat implementations of programming languages as defining their semantics, it is helpful not to do this, since implementations of interpreters and compilers are themselves programs and can be buggy.

Furthermore, different implementations of the same programming language may exhibit *diverging behaviors*, especially in the presence of under-specification. We can see that it is helpful to know what programs are meant to do (i.e., their semantics) separately from their implementations.

1.2 Formal Syntax Rules

1.2.1 Formal Language

A formal language L over an alphabet Σ is a subset of Σ^* , which is the set of strings consisting of and making use of elements in Σ . The empty string is denoted by ε .

For example, a formal language L over $\Sigma = \{1, 2\}$ is a subset of Σ^* , e.g. $L = \{\varepsilon, 1, 2, 11, 22, 111, 222\}$.

One way to manipulate strings is using a rewriting system, which consists of rules that map strings in a formal language to other strings in the language. For a language L , we define a relation $\rightarrow$ over strings in L . For example, for $L = \{1, 2, 11\}$, we may have $\rightarrow = \{(1, 2), (2, 1), (1, 11), (11, 2)\}$. Then we can write $1 \rightarrow 2$, $2 \rightarrow 1$, $1 \rightarrow 11$, and $11 \rightarrow 2$.

We also have generative grammar, which is a kind of rewriting system over a language $(\Sigma \cup N)^*$, where

- the alphabet is partitioned into two sets: a set of non-terminal symbols N and a set of terminal symbols Σ ,

- there is a distinguished start symbol in N ,

- rewrite rules are called *production rules*; the left-hand side of each rule contains at least one non-terminal.

We can use generative grammars to define a language over the alphabet Σ . The language defined is the set of strings of terminal symbols that we can generate by starting with the start symbol S and following the production rules.

Example (Generative Grammar). Consider $\Sigma \equiv \{a, b, c\}$.

Language ONE with only two strings $\{a, b\}$.

In this case, to generate a language over Σ , we have $N = \{S\}, \Sigma = \{a, b, c\}$ with grammar (production rules):

$$S \rightarrow a$$

$$S \rightarrow b$$

Language A-B where no a follows any b , e.g. $\varepsilon, ab, aab, acb$.

In this case, consider $N = \{S, A, B, C\}, \Sigma = \{a, b, c\}$ with production rules:

$$S \rightarrow CAB C$$

$$C \rightarrow \varepsilon \quad C \rightarrow cC$$

$$A \rightarrow \varepsilon \quad A \rightarrow aCA$$

$$B \rightarrow \varepsilon \quad B \rightarrow bCB$$

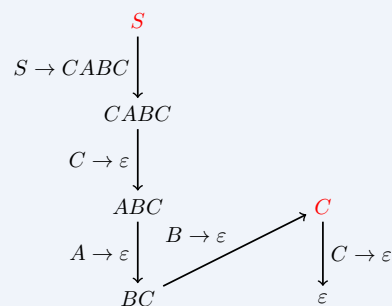
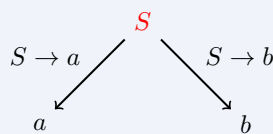
Language MATCH where the number of a s and b s is the same.

In this case, consider $N = \{S\}, \Sigma = \{a, b, c\}$ with production rules:

$$S \rightarrow \varepsilon \quad S \rightarrow c$$

$$S \rightarrow aSb \quad S \rightarrow bSa$$

$$S \rightarrow SS$$



1.2.2 Context-Free Grammar (CFGs)

The above grammars are context-free grammars, where the left-hand side of each production rule consists of a single non-terminal. It is often specified using Backus-Naur Form (BNF), where

- every production rule is written with $::=$ instead of $\rightarrow$,

- production rules with the same left-hand side are combined into a single rule, where different right-hand sides are separated with $|$, e.g. Language BOOL of boolean formulas with

- conjunction (and) $\wedge$
- disjunction (or) $\vee$
- negation (not) $\neg$

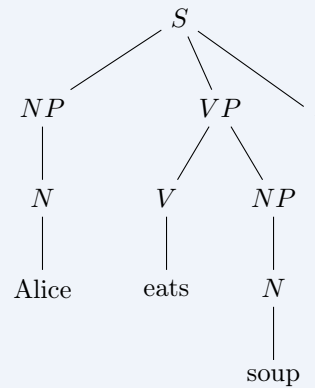
$$S ::= x \mid y \mid z \mid w \mid (S) \mid S \wedge S \mid S \vee S \mid \neg S$$

Example (BNF). Consider a BNF grammar for LUNCH⁻.

$$\begin{aligned} S &::= NP \ VP. \\ NP &::= N \mid N \ PP \\ VP &::= V \mid V \ NP \mid V \ PP \mid V \ NP \ PP \\ V &::= \text{buys} \mid \text{eats} \\ N &::= \text{Alice} \mid \text{Bob} \mid \text{soup} \mid \text{salad} \mid \text{pizza} \\ PP &::= \text{in a bowl} \mid \text{in a teacup} \mid \text{in a house} \end{aligned}$$

Then we can derive a derivation tree (parse tree):

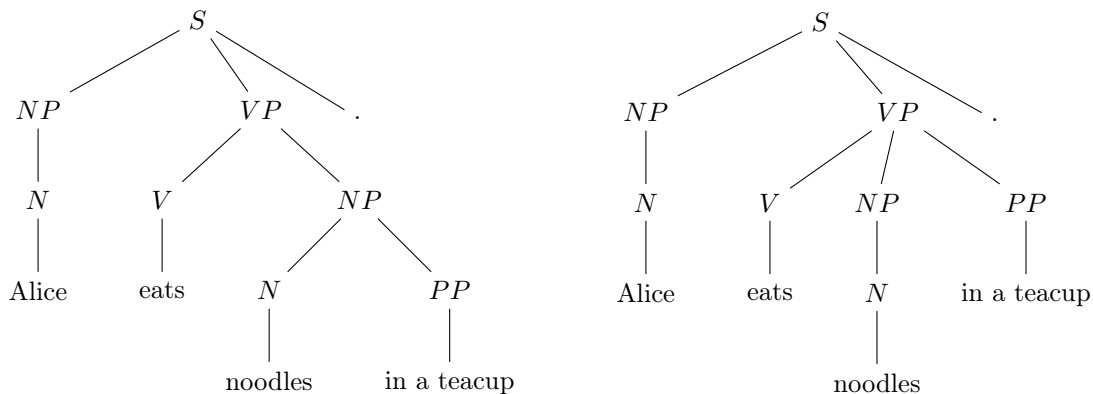
$$\begin{aligned} S &\rightarrow NP \ VP. \\ &\rightarrow N \ VP. \\ &\rightarrow \text{Alice} \ VP. \\ &\rightarrow \text{Alice} \ VNP. \\ &\rightarrow \text{Alice} \ \text{eats} \ NP. \\ &\rightarrow \text{Alice} \ \text{eats} \ N. \\ &\rightarrow \text{Alice} \ \text{eats} \ \text{soup}. \end{aligned}$$



Remark. A parse tree acts as a proof that a string is in a language (membership).

1.2.3 Ambiguity & Dangling Else Problem

A grammar is ambiguous if the same string has more than one parse tree, e.g.



Grammar serves an additional purpose, that is, it can be used to give structure to a string in a language. Semantics for the strings within a language are typically described with respect to this additional structure imposed on the string. Ambiguous grammar can thus lead to ambiguous meanings being assigned to strings.

For example, in the above examples, the left suggests that the noodles are in a teacup, whereas the right one suggests that Alice is located in a teacup. While one interpretation may be more likely than another and judgment can be based on semantics and background knowledge, the correct conclusion is not determined by the grammar alone.

This problem can manifest in the *dangling else problem* in a programming language context.

Consider language IF with the following grammar:

$$\begin{aligned} S &::= C \mid \text{if } B \text{ then } S \mid \text{if } B \text{ then } S \text{ else } S \\ B &::= \text{true} \mid \text{false} \mid \dots \\ C &::= \dots \end{aligned}$$

To deal with ambiguity, we can

1. change the grammar such that it is **not** ambiguous, i.e. same language with a different grammar.

For example, for BOOL:

$$S ::= x \mid y \mid z \mid w \mid (S) \mid S \wedge S \mid S \vee S \mid \neg S$$

we change to

$$\begin{aligned} S &::= D \vee S \mid D \\ D &::= C \wedge D \mid C \\ C &::= (S) \mid \neg C \mid x \mid y \mid z \mid w \end{aligned}$$

2. change the syntax, i.e. a different language with a different grammar.

For example, we change language IF to

$$S ::= C \mid \text{if } B \text{ then } S \text{ end if} \mid \text{if } B \text{ then } S \text{ else } S \text{ end if}$$

1.3 Abstract syntax trees (ASTs)

Before, we have $\text{Interpreter}_{\mathcal{L}}$ for $\mathcal{L}$ that takes a program in $\mathcal{L}$ and produces a value. Typically, there is a frontend containing a *parser* and a backend containing an *evaluator*. The parser parses the input string according to a grammar and converts the input program into an intermediate representation called an abstract syntax tree (AST),

$$\text{Parser}_{\mathcal{L}} : \mathcal{L} \rightarrow \text{AST}(\mathcal{L})$$

and the evaluator then evaluates the AST to a value,

$$\text{Evaluator}_{\text{AST}(\mathcal{L})} : \text{AST}(\mathcal{L}) \rightarrow \mathcal{V}$$

Then we have

$$\text{Interpreter}_{\mathcal{L}} = \text{Evaluator}_{\text{AST}(\mathcal{L})} \circ \text{Parser}_{\mathcal{L}}$$

The AST representation abstracts a parse tree and keeps only the relevant syntactic information from the parse tree, in particular, a subset of the production rules applied to obtain the parse tree.

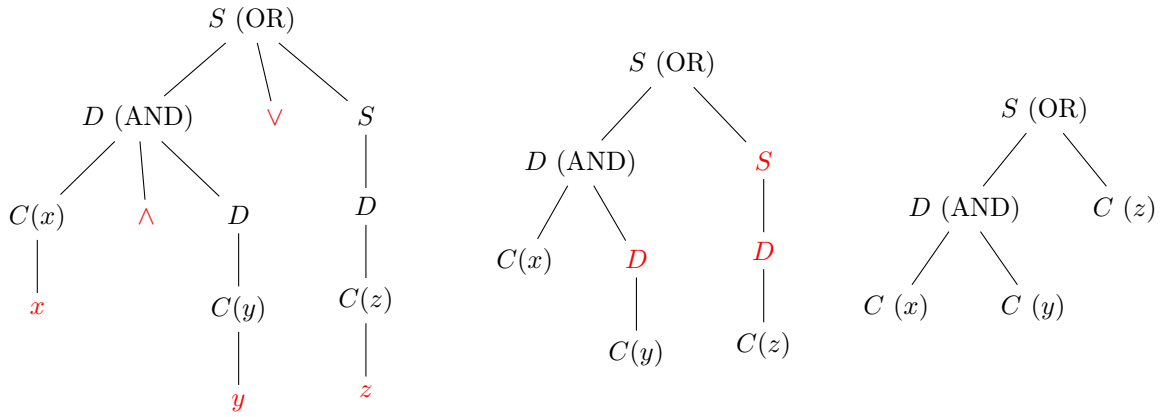
To make an AST:

1. derive a parse tree annotated with (important) production rules applied,
2. remove terminal nodes,
3. remove nodes expanded by production rules we do not care about; on removal, parent the node's children to its parent,
4. relabel non-terminal nodes with the annotated production rule names.

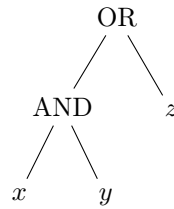
For example, for $x \wedge y \vee z$ with

$$\begin{aligned} S &::= D \vee S \mid D \\ D &::= C \wedge D \mid C \\ C &::= (S) \mid \neg C \mid x \mid y \mid z \mid w \end{aligned}$$

we have



finally we have



Notice that we have viewed ASTs as generic trees, but they are often trees with a specific structure, i.e. nodes with a certain label might always have a particular degree or children that have only certain kinds of labels. This structure can be captured by another grammar, where we introduce a production rule in the AST grammar for each production rule that we care about. For example, for the above BOOL, we might have the following grammar:

$$AST ::= OR(AST, AST) \mid AND(AST, AST) \mid NOT(AST) \mid x \mid y \mid z \mid w$$

This can be directly translated into an ADT, such as the following in OCaml:

```
1 type bool_ast = Or of bool_ast * bool_ast
2   | And of bool_ast * bool_ast
3   | Not of bool_ast
4   | X
5   | Y
6   | Z
7   | W
```

The `bool_ast` for $x \wedge y \vee z$ is `Or(And(X,Y),Z)`.

When implementing a parser, rather than generating a full parse tree and then an AST, we can generate the AST during parsing by applying the appropriate ADT constructor for a production rule.

1.4 Beyond BNF and Meta-variables

Many programming languages use formal grammars to specify syntax, yet they do not directly use BNF. Instead, they use **extensions** or **variants** of BNF.

For example, in the language BOOL, we have

$$Vars \cup \{\wedge, \vee, \neg, (,)\}$$

and in the grammar each variable in $Vars$ becomes a terminal in the production rules. To extend BOOL such that it allows more variables requires updating the BNF grammar with new production rules. However, when we want to allow infinitely many variable names, this becomes impossible as BNF grammars are finite.

We then use extensions of BNF which allow using additional notations outside of BNF, e.g. *meta-variables*. For example, let *BoolVars* be an infinite set of variables and let *v* range over *BoolVars*; then *v* is a stand-in for any element of *BoolVars*. Then we can still construct parse trees using BNF-like grammars and derive ASTs for them as before.

For example, consider the BNF grammar:

$$\begin{aligned} S &::= Z \mid Z + S \\ Z &::= -N \mid N \\ N &::= DN \mid D \\ D &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

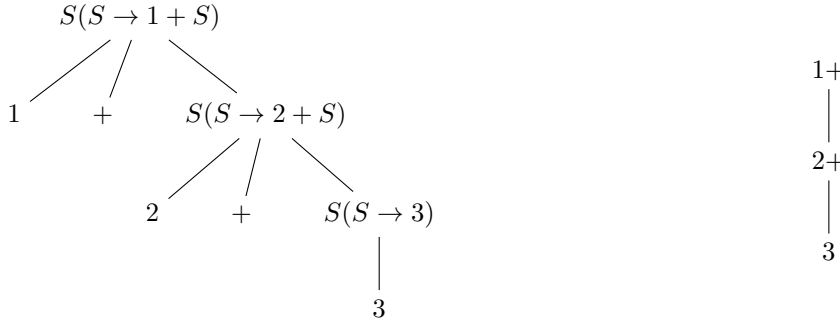
We typically write this as:

For *z* ranging over integers:

$$S ::= z \mid z + S$$

Here *z* is an integer, which is a semantic constraint, and this form of grammar is technically an extension of BNF.

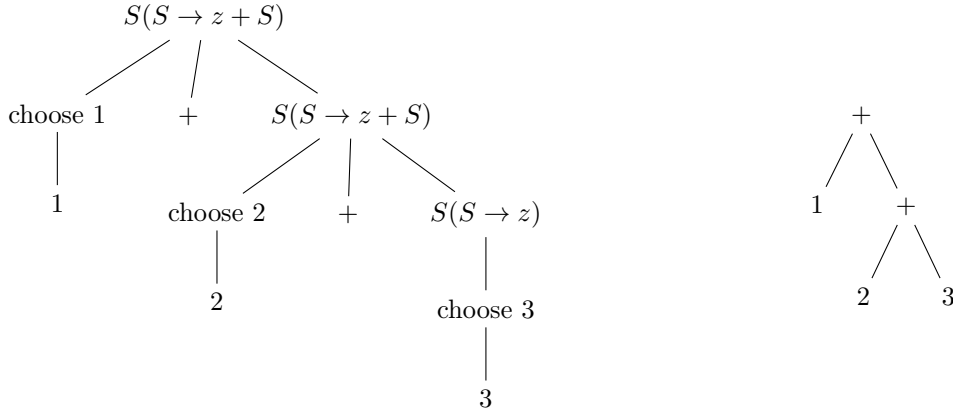
For example, we have a parse tree and AST for $1 + 2 + 3$ as:



However, consider tracking chosen *z* values, treating *z* as if generated by an infinite set of production rules:

$$z \in Z ::= \dots \mid -1 \mid 0 \mid 1 \mid \dots$$

Then we have

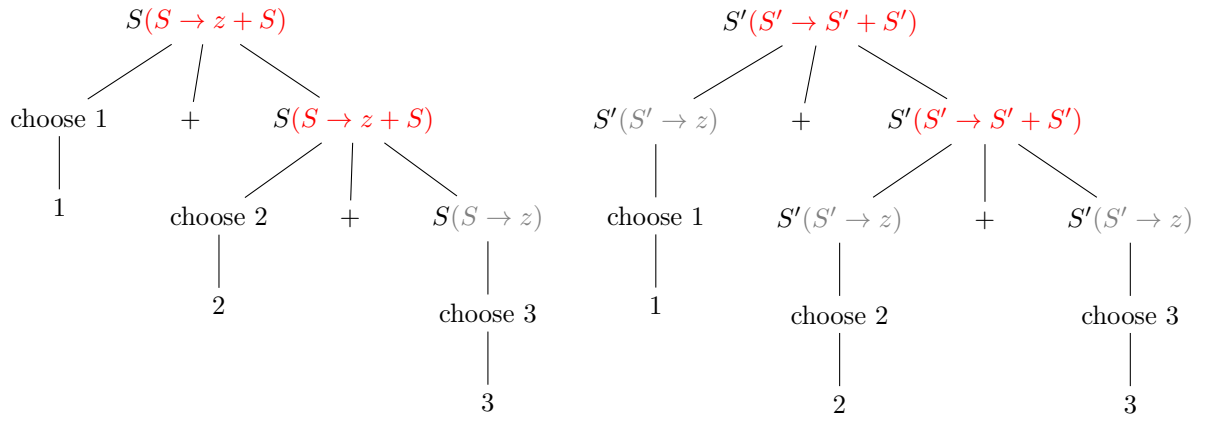


Note that an AST is an abstraction of the original parse tree that retains semantically meaningful parts. Choices of meta-variables are usually important.

For the above example, there is another grammar that generates the same language, i.e.

$$S' ::= z \mid S' + S'$$

For example, we have



which give the same AST.

So consider $S' ::= z \mid S' + S'$. If $+$ is left-associative, then the left-most $+$ will bind most tightly. We then expect this $+$ to be closest to the leaves in the AST, and the left operation is evaluated first.

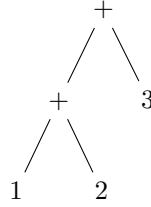


Figure 1.1: Left Associative

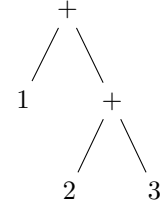


Figure 1.2: Right Associative

To further reduce ambiguity, we can add a meta-variable a that ranges over strings generated by the grammar $S' ::= z \mid S' + S'$, i.e.

$$a \in S' ::= z \mid a_1 + a_2$$

where $+$ is right-associative (by declaring the type of associativity to reduce ambiguity).

Then consider **BOOL** again. We can use meta-variables and BNF-like syntax to allow variable names to be anything in the possibly infinite set *BoolVars*, e.g.

$$S ::= v \mid (S) \mid S \wedge S \mid S \vee S \mid \neg S$$

and to remove ambiguity, we can specify the *precedence* and *associativity* of $\wedge, \vee, \neg$.

The OCaml ADT for the structure of the ASTs for **BOOL**⁺ could be something like

```
1 | type bool_ext_ast = Or of bool_ext_ast * bool_ext_ast
2 |   | And of bool_ext_ast * bool_ext_ast
3 |   | Not of bool_ext_ast
4 |   | Var of string
```

Chapter 2

Formal Proof Systems

In the previous chapter we mentioned that we can prove the membership of a string in a language by giving a parse tree. For example, for a language Nat with start symbol N and terminals $z, s, (,)$, we have

$$N ::= z \mid s(N)$$

and to prove $s(s(z)) \in \mathbf{Nat}$, we have

$$\begin{aligned} N &\rightarrow s(N) \\ &\rightarrow s(s(N)) \\ &\rightarrow s(s(z)) \end{aligned}$$

Then, how do we perform reasoning in a completely syntactic way rather than semantic? We use a proof system.

2.1 Proof Systems

Proof systems give us a way of performing reasoning in a completely syntactic way. A proof system has three components:

1. a formal language L of statements,
2. a set of inference rules (aka proof rules) that show how to prove statements. These typically have the following form, where the *premises* have zero or more statements and the *conclusion* is a single statement:

$$\frac{\text{premises}}{\text{conclusion}}$$

3. a subset of the inference rules called *axioms*, whose sets of premises are empty — we may omit the horizontal line when writing such rules.

2.1.1 Propositional Logic

Statements of propositional logic range over a language consisting of propositional variables and connectives, where propositional variables represent propositions, which may either be true ($\top$) or false ($\perp$).

Consider the grammar for **propositional logic**:

$$\begin{aligned} P &::= A \mid P \overset{\text{binary}}{\odot} P \mid P \overset{\text{not}}{\neg} P \mid \overset{\text{true}}{\top} \mid \overset{\text{false}}{\perp} \\ \odot &::= \underbrace{\wedge}_{\text{and}} \mid \underbrace{\vee}_{\text{or}} \mid \underbrace{\Rightarrow}_{\text{implies}} \end{aligned}$$

Capital letters like A, B, C range over a set of propositional variables. For example, we can have $A \Rightarrow A$, $A \wedge B \Rightarrow A$, $A \wedge B \vee C$, $A \wedge B \Rightarrow A \vee B$.

While the semantics of a programming language is defined in terms of its abstract syntax (like ASTs) rather than concrete syntax, we typically write the concrete syntax in place of the abstract syntax and indicate the structure in the AST using parentheses when it is ambiguous. For example, we write $(A \vee B) \wedge C$ rather than $\text{AND}(\text{OR}(A, B), C)$.

To remove ambiguity, we assume:

1. $\wedge$ has precedence over $\vee$ and $\Rightarrow$, and $\vee$ has precedence over $\Rightarrow$,
2. $\neg$ has precedence over all other connectives,
3. all connectives other than $\Rightarrow$ are left-associative,
4. $\Rightarrow$ is right-associative.

Note that “has precedence over” means “binds more tightly”, e.g. $A \wedge B \vee C$ is interpreted as $(A \wedge B) \vee C$.

As for associativity, for left-associative operators, the further to the left an operator is, the more tightly it binds, e.g. $A \wedge B \wedge C \wedge D$ is treated as $((A \wedge B) \wedge C) \wedge D$.

2.1.2 Natural Deduction System

Again we consider propositional logic:

$$\begin{aligned} S &::= [P] \mid P \\ P &::= A \mid P \odot P \mid \neg P \mid \top \mid \perp \\ \odot &::= \wedge \mid \vee \mid \Rightarrow \end{aligned}$$

where P is a propositional statement, and $[P]$ is an assumption. Therefore, plain P is the conclusion, and $[P]$ is an assumption during the proof.

Here let P, Q, R be meta-variables ranging over arbitrary propositional formulas. For each logical connective, there are *introduction* rules (which introduce the connective in the conclusion) and *elimination* rules (which eliminate the connective from one of the premises in the conclusion). Then we have:

- Truth and Falsity

$$\text{(UNIT)} \frac{}{\top} \qquad \text{(EX FALSO)} \frac{\perp}{P}$$

For introduction, $\top$ is always true. For elimination, if you have $\perp$ (false / contradiction), you can conclude any proposition P (provable but not necessarily true).

- Negation

$$\text{(\neg INTRO)} \frac{\begin{array}{c} [P] \\ \vdots \\ \perp \end{array}}{\neg P} \qquad \text{(\neg ELIM)} \frac{P \quad \neg P}{\perp}$$

For introduction, if assuming P leads to a contradiction, then you can conclude $\neg P$. For elimination, if both P and $\neg P$ exist, you get a contradiction.

- Conjunction

$$\text{(\wedge INTRO)} \frac{P \quad Q}{P \wedge Q} \qquad \text{(\wedge ELIM 1)} \frac{P \wedge Q}{P} \qquad \text{(\wedge ELIM 2)} \frac{P \wedge Q}{Q}$$

- Disjunction

$$\text{(\vee INTRO 1)} \frac{P}{P \vee Q} \qquad \text{(\vee INTRO 2)} \frac{Q}{P \vee Q} \qquad \text{(\vee ELIM)} \frac{P \vee Q \quad \begin{array}{c} [P] \\ \vdots \\ R \end{array} \quad \begin{array}{c} [Q] \\ \vdots \\ R \end{array}}{R}$$

- **Implication**

$$(\Rightarrow \text{INTRO}) \frac{\begin{array}{c} [P] \\ \vdots \\ Q \end{array}}{P \Rightarrow Q} \quad (\text{MODUS PONENS}) \frac{P \quad P \Rightarrow Q}{Q}$$

- **Assumption**

$$\frac{\begin{array}{c} [P] \\ \vdots \\ P \end{array}}{P}$$

Then we can formalize an informal proof:

Example. Consider

If Alice likes tea and Alice likes bread, then Alice likes tea.

So to prove $T \wedge B \Rightarrow T$, we consider

$$(\Rightarrow \text{INTRO}) \frac{\begin{array}{c} [T \wedge B] \\ \boxed{?} \\ T \end{array}}{T \wedge B \Rightarrow T}$$

using assumption, we have

$$(\text{ASSUMPTION}) \frac{\begin{array}{c} [T \wedge B] \\ T \wedge B \\ \boxed{?} \\ T \end{array}}{T \wedge B \Rightarrow T}$$

using $\wedge\text{ELIM 1}$, we have

$$(\wedge\text{ELIM 1}) \frac{\begin{array}{c} [T \wedge B] \\ T \wedge B \\ T \end{array}}{T \wedge B \Rightarrow T}$$

This is a derivation tree (proof tree).

We can modify the form of statements to keep track of assumptions in a context Γ , which is a comma-separated list of propositional logic statements P that we treat as assumptions, i.e.

$$S ::= \Gamma \vdash P$$

$$\Gamma ::= \Gamma, P \mid \emptyset$$

Here Γ ranges over possible contexts (lists of assumptions). Then we have

$$\begin{array}{l} (\text{ASSUMPTION}) \frac{}{\Gamma \vdash P} \text{ if } P \text{ is in } \Gamma \\ (\text{UNIT}) \frac{}{\Gamma \vdash \top} \quad (\text{EX FALSO}) \frac{\Gamma \vdash \perp}{\Gamma \vdash P} \quad (\neg \text{INTRO}) \frac{\Gamma, P \vdash \perp}{\Gamma \vdash \neg P} \quad (\neg \text{ELIM}) \frac{\Gamma \vdash P \quad \Gamma \vdash \neg P}{\Gamma \vdash \perp} \\ (\wedge \text{INTRO}) \frac{\Gamma \vdash P \quad \Gamma \vdash Q}{\Gamma \vdash P \wedge Q} \quad (\wedge \text{ELIM 1}) \frac{\Gamma \vdash P \wedge Q}{\Gamma \vdash P} \quad (\wedge \text{ELIM 2}) \frac{\Gamma \vdash P \wedge Q}{\Gamma \vdash Q} \\ (\Rightarrow \text{INTRO}) \frac{\Gamma, P \vdash Q}{\Gamma \vdash P \Rightarrow Q} \quad (\text{MODUS PONENS}) \frac{\Gamma \vdash P \quad \Gamma \vdash P \Rightarrow Q}{\Gamma \vdash Q} \\ (\vee \text{INTRO 1}) \frac{\Gamma \vdash P}{\Gamma \vdash P \vee Q} \quad (\vee \text{INTRO 2}) \frac{\Gamma \vdash Q}{\Gamma \vdash P \vee Q} \\ (\vee \text{ELIM}) \frac{\Gamma \vdash P \vee Q \quad \Gamma, P \vdash R \quad \Gamma, Q \vdash R}{\Gamma \vdash R} \end{array}$$

Note that the typical notation omits $\emptyset$ when writing Γ . For example, instead of writing $\emptyset \vdash \top$, we write $\vdash \top$.

2.2 From Grammars to Proof System

Recall language Nat with start symbol N and terminals z, s :

$$N ::= z \mid s(N)$$

where we have an equivalent set of inference rules:

$$\text{ZERO } \frac{}{z} \quad \text{SUCC } \frac{N}{s(N)}$$

So to prove $s(s(z)) \in \text{Nat}$, we consider

$$\begin{aligned} N &\rightarrow s(N) \\ &\rightarrow s(s(N)) \\ &\rightarrow s(s(z)) \end{aligned}$$

or

$$\frac{\frac{\frac{}{z} \text{ (ZERO)}}{s(z)} \text{ (SUCC)}}{s(s(z))} \text{ (SUCC)}$$

We can also convert production rules to inference rules with notation $e : E$, i.e. e can be generated from E . Again recall the language BOOL:

$$\begin{aligned} S &::= D \vee S \mid D \\ D &::= C \wedge D \mid C \\ C &::= (S) \mid \neg C \mid x \mid y \mid z \mid w \end{aligned}$$

We have

$$\begin{aligned} \text{(OR)} \quad & \frac{d : D \quad s : S}{d \vee s : S} \quad \frac{d : D}{d : S} \quad \text{(AND)} \quad \frac{c : C \quad d : D}{c \wedge d : D} \quad \frac{c : C}{c : D} \\ & \frac{s : S}{(s) : C} \quad (x) \frac{}{x : C} \quad (y) \frac{}{y : C} \quad (z) \frac{}{z : C} \quad (w) \frac{}{w : C} \end{aligned}$$

Then we can:

1. prove that the string can be generated by S using the proof rules,
2. construct a tree by making a node for each important derivation step, where a node's first named descendant in the tree is its child.

For example, we can have:

$$\frac{\frac{\frac{}{x : C} (x)}{x : D} \quad \frac{\frac{\frac{}{y : C} (y)}{y : D}}{y : S} \text{ (OR)}}{x \vee y : S}$$

2.3 Predicate Logic

Instead of propositional variables, we have:

- predicate symbols, which are applied to zero or more terms,
- function symbols, which are applied to zero or more terms,
- quantifiers $\forall, \exists$, which are applied to variables.

A term is either a variable or an n -ary function symbol applied to n terms, e.g.

$$\forall x. 1 + x < 0 \quad \text{and} \quad \forall x. \forall y. \forall z. x > y \wedge y > z \Rightarrow x > z$$

Then for

If Alice likes tea and Alice likes bread, then Alice likes tea.

we have

$$\text{Likes}(\text{Alice}, \text{tea}) \wedge \text{Likes}(\text{Alice}, \text{bread}) \Rightarrow \text{Likes}(\text{Alice}, \text{tea})$$

For

If Alice likes tea and tea is in the bag, then Alice likes something in the bag.

we have

$$\text{Likes}(\text{Alice}, \text{tea}) \wedge \text{InBag}(\text{tea}) \Rightarrow \exists \text{thing}. \text{Likes}(\text{Alice}, \text{thing}) \wedge \text{InBag}(\text{thing})$$

Example (AST and Proof Rules). Consider Language ADDITION. We add a meta-variable a that ranges over strings generated by the grammar $S ::= z \mid S' + S'$:

$$a \in S' ::= z \mid a_1 + a_2 \quad \text{where } + \text{ is right-associative}$$

We then try to define an “evaluation step” with a proof system whose statements (judgments) look like

$$a_1 \rightarrow a_2$$

We have the inference rules:

$$\text{(PLUS)} \quad z_1 + z_2 \rightarrow z_3 \quad \text{if } z_3 = z_1 + z_2$$

$$\text{(LEFT)} \quad \frac{a_1 \rightarrow a'_1}{a_1 + a_2 \rightarrow a'_1 + a_2}$$

$$\text{(RIGHT)} \quad \frac{a \rightarrow a'}{z + a \rightarrow z + a'}$$

The program will then evaluate to an integer z . Now consider evaluating $1 + 2 + 3$.

For PLUS, we have $z_1 + z_2$; however, we cannot instantiate meta-variables in $z_1 + z_2$ to get $1 + 2 + 3$, since both z_1 and z_2 must be integers (single-node ASTs).

However, for LEFT and RIGHT, we can match $1 + 2 + 3$. For LEFT, we have $a_1 + a_2$, where a_i ranges over any expression in the language, so any sub-AST can replace a_i . This is the same for RIGHT.

Consider LEFT. After instantiating $a_1 = 1$ and $a_2 = 2 + 3$, we have

$$\frac{1 \rightarrow a'_1}{1 + 2 + 3 \rightarrow a'_1 + 2 + 3}$$

To finish this proof, we need to prove the premise $1 \rightarrow a'_1$ for some instantiation of a'_1 . However, no rule applies to 1, since it is already a value, so the proof fails.

Next, consider RIGHT:

$$\frac{2 + 3 \rightarrow a'}{1 + 2 + 3 \rightarrow 1 + a'}$$

To finish the proof, we need to prove $2 + 3 \rightarrow a'$. This does not fail; instead, we must continue evaluating $2 + 3$.

Now we can apply PLUS by instantiating $z_1 = 2$ and $z_2 = 3$:

$$2 + 3 \rightarrow z_3 \quad \text{if } 2 + 3 = z_3$$

From the side condition, $z_3 = 5$, so $a' = 5$. Hence,

$$\frac{2 + 3 \rightarrow 5}{1 + 2 + 3 \rightarrow 1 + 5}$$

We then evaluate $1 + 5$, which is similar to $2 + 3$. By applying PLUS, we obtain:

$$1 + 5 \rightarrow 6$$

Chapter 3

Semantics Basics

3.1 Introduction to Semantics

Consider the following C++14 program:

```
1 int main (int argc, char *argv[]) {  
2     int i = 1;  
3     int v = 2;  
4     i = i++ + v;  
5     printf("%d\n", i);  
6  
7     return 0;  
8 }
```

Using the `clang` compiler it prints 3, while using the `VS` compiler, it prints 4. However, what should it do and what is it supposed to do? The answer comes from the semantics of programming languages.

Most programming languages have semantics described in natural language, which is preferable for many purposes but can be *imprecise* and *ambiguous*. For example, consider a programming language with a built-in function `sort`, where the documentation says:

The `sort` function takes an array and a comparator function as arguments. It sorts the array using the comparator function and returns it.

This description is *imprecise*, as there is no information on whether it modifies the original array or creates a new one, nor even the order of sorting.

We can modify the description to be more precise and less ambiguous, yet natural language itself is hard to make fully precise. The ambiguity and imprecision of natural language can lead to diverging implementations of the same programming language, which leads to unexpected behavior.

Therefore, we want formal semantics that give meaning to programs in a **precise**, **unambiguous**, and **concise** way.

Different ways include:

- **Operational semantics**: how is a program executed on an abstract machine?
- **Denotational semantics**: what does the program mean in terms of mathematical objects?
- **Axiomatic semantics**: what should we be able to prove about this program?

3.2 SIMPL

SIMPL is the Simple IMPerative Language. This is an *imperative* programming language, where programs are expressed in terms of sequences of commands or statements. It has common imperative control structures, such as while loops and if-statements. It also has *locations* that hold integer or boolean values,

which can be *dereferenced* using the `!` operator.

The grammar of SIMPL is as below:

$$\begin{aligned}
 e \in \langle \text{expr} \rangle &::= b \mid z \mid \mathbf{skip} \\
 &\mid r := e && \text{assignment} \\
 &\mid !r && \text{dereference} \\
 &\mid e_1; e_2 && \text{sequencing} \\
 &\mid \mathbf{if} \ e_1 \ \mathbf{then} \ e_2 \ \mathbf{else} \ e_3 && \text{if statement} \\
 &\mid \mathbf{while} \ e_1 \ \mathbf{do} \ e_2 && \text{while loop} \\
 &\mid e_1 \odot e_2 \\
 \odot \in \langle \text{op} \rangle &::= + \mid - \mid * \mid / \mid \leq
 \end{aligned}$$

Here r ranges over a set of allowed location names, b ranges over the set of Booleans $\mathbb{B}$, and z ranges over the set of integers $\mathbb{Z}$.

We remove ambiguity by making the following assumptions:

1. $*$ and $/$ have precedence over $+$ and $-$, so that $e_1 + e_2 \odot e_3$ is interpreted as $e_1 + (e_2 \odot e_3)$, where $\odot \in \{*, /\}$,
2. operators $+$, $-$, $*$, $/$ all bind more tightly than $\leq$,
3. the sequencing operator $;$ is right-associative and binds the least tightly,
4. operators $\odot$ are left-associative.

For SIMPL, we define the set of values that the program evaluates to by

$$v \in \text{Values} ::= z \mid b \mid \mathbf{skip}$$

Example.

Program 1:

```

1 | x := 3;
2 | y := 1;
3 | while 2 < !x do (y := !x * !y; x := !x - 1)

```

Program 2:

```

1 | x := 3;
2 | z := (y := 10; if !y ≤ !x then !x else !y);
3 | !z

```

Program 3:

```

1 | x := 1;
2 | while 0 ≤ !x do x := !x + 1

```

Program 4:

```

1 | x := 1;
2 | while !x do x := !x - 1

```

3.3 Small-step Operational Semantics

Since programs describe computations, we can think of the meaning of a program as the computation it describes. This is essentially *operational semantics*, where the meaning of programs is defined by describing how a program should be executed on an abstract machine.

This definition is given via a proof system in which rules' statements (judgments) describe the execution steps that the abstract machine can perform. If we can prove the steps can be taken, we can take them.

In small-step operational semantics, judgments describe a *single* atomic computation step that the abstract machine can perform. Each step is called a *reduction*. Evaluating a program is then performing reductions until a value is reached.

In SIMPL, the judgments in particular will involve reducing *configurations*, which are pairs of a program and a program *state* that keeps track of the context in which the program should be executed. For SIMPL we use a *store*, which is a map from location names to the values stored in them. For example:

$$\{\text{var1} \mapsto 1, \text{var2} \mapsto 2\}$$

where locations **var1** and **var2** store values 1 and 2 respectively.

This store is a *mutable store* in which the values may change.

The operational semantics rules for SIMPL are as follows:

$$\begin{aligned}
&(\text{ASSIGN-BOOL}) (r := b, s) \rightarrow (\text{skip}, s[r \mapsto b]) & (\text{DEREF-BOOL}) (!r, s) \rightarrow (b, s) \text{ if } s[r] = b \\
&(\text{ASSIGN-INT}) (r := z, s) \rightarrow (\text{skip}, s[r \mapsto z]) & (\text{DEREF-INT}) (!r, s) \rightarrow (z, s) \text{ if } s[r] = z \\
&(\text{ASSIGN-EXPR}) \frac{(e, s) \rightarrow (e', s')}{(r := e, s) \rightarrow (r := e', s')} \\
&(\text{SEQ-1}) (\text{skip}; e, s) \rightarrow (e, s) & (\text{SEQ-2}) \frac{(e_1, s) \rightarrow (e'_1, s')}{(e_1; e_2, s) \rightarrow (e'_1; e_2, s')} \\
&(\text{IF-TRUE}) (\text{if true then } e_2 \text{ else } e_3, s) \rightarrow (e_2, s) \\
&(\text{IF-FALSE}) (\text{if false then } e_2 \text{ else } e_3, s) \rightarrow (e_3, s) \\
&(\text{IF-EXPR}) \frac{(e_1, s) \rightarrow (e'_1, s')}{(\text{if } e_1 \text{ then } e_2 \text{ else } e_3, s) \rightarrow (\text{if } e'_1 \text{ then } e_2 \text{ else } e_3, s')} \\
&(\text{WHILE}) (\text{while } e_1 \text{ do } e_2, s) \rightarrow (\text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else skip}, s) \\
&(\text{OP-1}) \frac{(e_1, s) \rightarrow (e'_1, s')}{(e_1 \odot e_2, s) \rightarrow (e'_1 \odot e_2, s')} & (\text{OP-2}) \frac{(e_2, s) \rightarrow (e'_2, s')}{(v \odot e_2, s) \rightarrow (v \odot e'_2, s')} \\
&(\text{LT}) (z_1 \leq z_2, s) \rightarrow (b, s) \text{ if } b = \underbrace{(z_1 \leq z_2)}_{\text{side condition}}
\end{aligned}$$

Note that there are some side conditions that must hold for some rules above to be applicable.

Then we can use these rules to do evaluation. Considering Program 1:

```

1 | x := 3;
2 | y := 1;
3 | while 2 < !x do (y := !x * !y; x := !x - 1)

```

Here we use Δ for $y := !x * !y$; $x := !x - 1$ and $\mathbf{W}$ for **while** $2 < !x$ **do** Δ . The initial configuration has the form $(e_1; e_2, \{\})$, where e_1 is $x := 3$ and e_2 is $y := 1$; $\mathbf{W}$.

In order to take a step to the next configuration, we need to make sure that the step is derivable using the operational semantics. We can reason about the structure of the program and the rules in the semantics to see if a step can be taken.

So in order to take a step from an expression of this form, we need a rule whose left-hand side has the form $(e_1; e_2, s)$, where e_1 is not **skip**:

$$(\text{SEQ-2}) \frac{(x := 3, \{\}) \rightarrow (e'_1, s')}{(x := 3; y := 1; \mathbf{W}, \{\}) \rightarrow (e'_1; e_2, s')}$$

To derive this we need to derive its premise. We choose to apply ASSIGN-INT:

$$(\text{ASSIGN-INT}) (x := 3, \{\}) \rightarrow (\text{skip}, \{x \mapsto 3\})$$

For ASSIGN-EXPR, we would need to take a step from $(3, s)$, but there is no rule for this judgment. Thus we have the full derivation tree:

$$\frac{\overline{(x := 3, \{\}) \rightarrow (\mathbf{skip}, \{x \mapsto 3\})} \text{ (ASSIGN-INT)}}{(x := 3; y := 1; \mathbf{W}, \{\}) \rightarrow (\mathbf{skip}; y := 1; \mathbf{W}, \{x \mapsto 3\})} \text{ (SEQ-2)}$$

We have the full evaluation as follow:

$$\begin{aligned} (x := 3; y := 1; \mathbf{W}, \{\}) &\rightarrow (\mathbf{skip}; y := 1; \mathbf{W} && , \{x \mapsto 3\}) \\ &\rightarrow (y := 1; \mathbf{W} && , \{x \mapsto 3\}) \\ &\rightarrow (\mathbf{skip}; \mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\mathbf{if } 2 \leq! x \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\mathbf{if } 2 \leq 3 \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\mathbf{if true then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\Delta; \mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (y := 3 *! y; x :=! x - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (y := 3 * 1; x :=! x - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (y := 3; x :=! x - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 1\}) \\ &\rightarrow (\mathbf{skip}; x :=! x - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 3\}) \\ &\rightarrow (x :=! x - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 3\}) \\ &\rightarrow (x := 3 - 1; \mathbf{W} && , \{x \mapsto 3, y \mapsto 3\}) \\ &\rightarrow (x := 2; \mathbf{W} && , \{x \mapsto 3, y \mapsto 3\}) \\ &\rightarrow (\mathbf{skip}; \mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\mathbf{if } 2 \leq! x \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\mathbf{if } 2 \leq 2 \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\mathbf{if true then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\Delta; \mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (y := 2 *! y; x :=! x - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (y := 2 * 3; x :=! x - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (y := 6; x :=! x - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 3\}) \\ &\rightarrow (\mathbf{skip}; x :=! x - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 6\}) \\ &\rightarrow (x :=! x - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 6\}) \\ &\rightarrow (x := 2 - 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 6\}) \\ &\rightarrow (x := 1; \mathbf{W} && , \{x \mapsto 2, y \mapsto 6\}) \\ &\rightarrow (\mathbf{skip}; \mathbf{W} && , \{x \mapsto 1, y \mapsto 6\}) \\ &\rightarrow (\mathbf{W} && , \{x \mapsto 1, y \mapsto 6\}) \\ &\rightarrow (\mathbf{if } 2 \leq! x \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 1, y \mapsto 6\}) \\ &\rightarrow (\mathbf{if } 2 \leq 1 \mathbf{ then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 1, y \mapsto 6\}) \\ &\rightarrow (\mathbf{if false then } (\Delta; \mathbf{W}) \mathbf{ else skip} && , \{x \mapsto 1, y \mapsto 6\}) \\ &\rightarrow (\mathbf{skip} && , \{x \mapsto 1, y \mapsto 6\}) \end{aligned}$$

3.4 Evaluation Order and Determinism

3.4.1 Determinism

The semantics we looked at for SIMPL are *deterministic*, meaning each program has only one possible behavior. For small-step operational semantics, this means that for every expression that can be reduced by the semantics, there is exactly one way to reduce it.

We also made choices like evaluation order to obtain deterministic semantics. For example, for the following left-to-right evaluation order:

$$\text{(OP-1)} \frac{(e_1, s) \rightarrow (e'_1, s')}{(e_1 \odot e_2, s) \rightarrow (e'_1 \odot e_2, s')} \quad \text{(OP-2)} \frac{(e_2, s) \rightarrow (e'_2, s')}{(v \odot e_2, s) \rightarrow (v \odot e'_2, s')}$$

We can instead use a right-to-left evaluation order:

$$\text{(OP-1)} \frac{(e_2, s) \rightarrow (e'_2, s')}{(e_1 \odot e_2, s) \rightarrow (e_1 \odot e'_2, s')} \quad \text{(OP-2)} \frac{(e_1, s) \rightarrow (e'_1, s')}{(e_1 \odot v, s) \rightarrow (e'_1 \odot v, s')}$$

Yet these yield the same results for all SIMPL programs that do not interact with the store.

Such programs have no *side effects*, and functions like these are called *pure* functions that have no side effects and have deterministic results. They can be regarded as implementations of mathematical functions.

Remark. Side effects include any read/write to any kind of state.

However, most general-purpose programming languages have side effects, which include things like updating memory and writing to files, leading to different results from different evaluation orders. In the context of SIMPL, the only side effects are store updates.

Most languages are implemented with a left-to-right evaluation order. Many also implement something called *short-circuiting* for boolean operators, where if the boolean value of the expression can be determined from evaluating only the left operand, the right one will not be evaluated.

3.4.2 Nondeterminism

We can include both left-to-right and right-to-left rules in the small-step operational semantics. Then we have a nondeterministic semantics, which may lead programs to have different observable behaviors. If we allow nondeterministic semantics, then a single program could potentially be evaluated to several different values. Then the interpreter should be a relation:

$$\text{Interpreter}_L \subseteq L \times V$$

where for a program e and value v :

$$(e, v) \in \text{Interpreter}_L \Leftrightarrow e \text{ can evaluate to } v \text{ according to the semantics.}$$

There are cases where nondeterministic semantics are desirable. For example, the generation of random numbers. Consider adding the ability to generate random integers in SIMPL via the expression **randInt**. We then have:

$$(\text{randInt}, s) \rightarrow (z, s)$$

Note that this does not enforce randomness, i.e. the value generated by **randInt** could always be the integer 1 or a pseudorandom number.

Another case where nondeterministic semantics might be accepted is for *concurrency*, where programs are made up of parallel compositions of different programs that run at the same time. This can speed up overall computation, yet leads to nondeterministic semantics. For example, we have:

$$(x := 1 \parallel y := 2); !y$$

It is common to model the semantics of concurrent programs using *interleaving semantics*, in which the semantics of a parallel composition of programs is defined to be the interleaving of the semantic steps of the composed programs. For example:

$$\begin{aligned} \text{(LEFT-STEP)} \quad & \frac{(e_1, s) \rightarrow (e'_1, s')}{(e_1 \parallel e_2, s) \rightarrow (e'_1 \parallel e_2, s')} & \text{(RIGHT-STEP)} \quad & \frac{(e_2, s) \rightarrow (e'_2, s')}{(e_1 \parallel e_2, s) \rightarrow (e_1 \parallel e'_2, s')} \\ \text{(LEFT-SKIP)} \quad & (\text{skip} \parallel e, s) \rightarrow (e, s) & \text{(RIGHT-SKIP)} \quad & (e \parallel \text{skip}, s) \rightarrow (e, s) \end{aligned}$$

Then the four evaluations for $(x := 1 \parallel y := 2); !y$, starting from the empty store $\{\}$, allowed by the semantics are as follows:

Evaluation 1:

$$\begin{aligned}
 ((x := 1 \parallel y := 2); !y, \{\}) &\rightarrow ((\mathbf{skip} \parallel y := 2); !y, \{x \mapsto 1\}) \\
 &\rightarrow (y := 2; !y, \{x \mapsto 1\}) \\
 &\rightarrow (\mathbf{skip}; !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (!y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (2, \{x \mapsto 1, y \mapsto 2\})
 \end{aligned}$$

Evaluation 2:

$$\begin{aligned}
 ((x := 1 \parallel y := 2); !y, \{\}) &\rightarrow ((\mathbf{skip} \parallel y := 2); !y, \{x \mapsto 1\}) \\
 &\rightarrow ((\mathbf{skip} \parallel \mathbf{skip}); !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (\mathbf{skip}; !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (!y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (2, \{x \mapsto 1, y \mapsto 2\})
 \end{aligned}$$

Evaluation 3:

$$\begin{aligned}
 ((x := 1 \parallel y := 2); !y, \{\}) &\rightarrow ((x := 1 \parallel \mathbf{skip}); !y, \{y \mapsto 2\}) \\
 &\rightarrow (x := 1; !y, \{y \mapsto 2\}) \\
 &\rightarrow (\mathbf{skip}; !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (!y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (2, \{x \mapsto 1, y \mapsto 2\})
 \end{aligned}$$

Evaluation 4:

$$\begin{aligned}
 ((x := 1 \parallel y := 2); !y, \{\}) &\rightarrow ((x := 1 \parallel \mathbf{skip}); !y, \{y \mapsto 2\}) \\
 &\rightarrow ((\mathbf{skip} \parallel \mathbf{skip}); !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (\mathbf{skip}; !y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (!y, \{x \mapsto 1, y \mapsto 2\}) \\
 &\rightarrow (2, \{x \mapsto 1, y \mapsto 2\})
 \end{aligned}$$

Note that while the semantics is nondeterministic, the result of the program is deterministic regardless of how the program is evaluated. However, this is not always the case for all programs.

3.5 Big-step Operational Semantics

An alternative to small-step operational semantics is *big-step* operational semantics (also called natural semantics), which describes the evaluation of a program to its final value in a single “big” step $\Downarrow$.

Big-step operational semantics here will have judgments of the form $\Gamma \vdash e \Downarrow (v, s)$, where Γ gives some kind of context in which we consider the program, e is the program, v is the program’s final value, and s is the program’s final store.

In the case of SIMPL, Γ is simply a store, so we have judgments like $s \vdash e \Downarrow (v, s')$. Judgments expressing the same thing can be shown in different notations, for example

$$s \vdash e \Downarrow v \vdash s' \qquad (e, s) \Downarrow (v, s')$$

Regardless of notation, what makes these big-step operational semantics judgments is that they show the entire evaluation of the program in a single step.

The big-step operational semantics for SIMPL programs are as follows:

$$\begin{aligned}
 (\text{INT}) \quad & s \vdash z \Downarrow (z, s) & (\text{BOOL}) \quad & s \vdash b \Downarrow (b, s) & (\text{SKIP}) \quad & s \vdash \mathbf{skip} \Downarrow (\mathbf{skip}, s) \\
 (\text{ASSIGN-BOOL}) \quad & \frac{s \vdash e \Downarrow (b, s')}{s \vdash r := e \Downarrow (\mathbf{skip}, s'[r \mapsto b])} & (\text{ASSIGN-INT}) \quad & \frac{s \vdash e \Downarrow (z, s')}{s \vdash r := e \Downarrow (\mathbf{skip}, s'[r \mapsto z])}
 \end{aligned}$$

$$\begin{aligned}
& (\text{DEREF-BOOL}) \quad s \vdash !r \Downarrow (b, s) \text{ if } s[r] = b & (\text{DEREF-INT}) \quad s \vdash !r \Downarrow (z, s) \text{ if } s[r] = z \\
& (\text{SEQ}) \quad \frac{s \vdash e_1 \Downarrow (\text{skip}, s') \quad s' \vdash e_2 \Downarrow (v, s'')}{s \vdash e_1; e_2 \Downarrow (v, s'')} \\
& (\text{IF-TRUE}) \quad \frac{s \vdash e_1 \Downarrow (\text{true}, s') \quad s' \vdash e_2 \Downarrow (v, s'')}{s \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow (v, s'')} & (\text{IF-FALSE}) \quad \frac{s \vdash e_1 \Downarrow (\text{false}, s') \quad s' \vdash e_3 \Downarrow (v, s'')}{s \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Downarrow (v, s'')} \\
& (\text{WHILE}) \quad \frac{s \vdash \text{if } e_1 \text{ then } (e_2; \text{while } e_1 \text{ do } e_2) \text{ else skip} \Downarrow (v, s')}{s \vdash \text{while } e_1 \text{ do } e_2 \Downarrow (v, s')} \\
& (\text{LT}) \quad \frac{s \vdash e_1 \Downarrow (z_1, s') \quad s' \vdash e_2 \Downarrow (z_2, s'')}{s \vdash e_1 \leq e_2 \Downarrow (b, s'')} \text{ if } b = (z_1 \leq z_2) \\
& (\text{PLUS}) \quad \frac{s \vdash e_1 \Downarrow (z_1, s') \quad s' \vdash e_2 \Downarrow (z_2, s'')}{s \vdash e_1 + e_2 \Downarrow (z, s'')} \text{ if } z = z_1 + z_2 \\
& (\text{MINUS}) \quad \frac{s \vdash e_1 \Downarrow (z_1, s') \quad s' \vdash e_2 \Downarrow (z_2, s'')}{s \vdash e_1 - e_2 \Downarrow (z, s'')} \text{ if } z = z_1 - z_2 \\
& (\text{TIMES}) \quad \frac{s \vdash e_1 \Downarrow (z_1, s') \quad s' \vdash e_2 \Downarrow (z_2, s'')}{s \vdash e_1 * e_2 \Downarrow (z, s'')} \text{ if } z = z_1 \times z_2 \\
& (\text{DIVIDE}) \quad \frac{s \vdash e_1 \Downarrow (z_1, s') \quad s' \vdash e_2 \Downarrow (z_2, s'')}{s \vdash e_1 / e_2 \Downarrow (z, s'')} \text{ if } z = \left\lfloor \frac{z_1}{z_2} \right\rfloor
\end{aligned}$$

To see what a configuration (e, s) evaluates to using big-step semantics, we need to find a value v and store s' such that we can derive a judgment $s \vdash e \Downarrow (v, s')$ using only the rules of the big-step operational semantics.

The difference is that in small-step semantics, the rules allow us to prove *which steps we can take* in order to compute a value, whereas in big-step semantics, we can prove that a program evaluates to a value directly.

We would like the big-step semantics to match the small-step semantics, i.e.

$$s \vdash e \Downarrow (v, s') \text{ iff } (e, s) \rightarrow^* (v, s')$$

where $\rightarrow^*$ is the reflexive transitive closure of $\rightarrow$.

Example. Consider $((1 + 2) * 3, \{\})$.

Small-step:

$$\begin{aligned}
((1 + 2) * 3, \{\}) & \rightarrow (3 * 3, \{\}) \\
& \rightarrow (9, \{\})
\end{aligned}$$

where we have

$$\frac{\overline{((1 + 2), \{\}) \rightarrow (3, \{\})}}{\overline{((1 + 2) * 3, \{\}) \rightarrow (3 * 3, \{\})}} \text{ (PLUS)} \quad \text{(OP-1)} \quad \frac{\overline{(3 * 3, \{\}) \rightarrow (9, \{\})}}{\overline{(3 * 3, \{\}) \rightarrow (9, \{\})}} \text{ (TIMES)}$$

Big-step

Consider

$$\Delta = \overline{\{\} \vdash 1 \Downarrow (1, \{\})} \text{ (INT)} \quad \square = \overline{\{\} \vdash 2 \Downarrow (2, \{\})} \text{ (INT)}$$

then we have

$$\frac{\frac{\Delta \quad \square}{\{\} \vdash 1 + 2 \Downarrow (3, \{\})} \text{ (PLUS)} \quad \overline{\{\} \vdash 3 \Downarrow (3, \{\})} \text{ (INT)}}{\{\} \vdash (1 + 2) * 3 \Downarrow (9, \{\})} \text{ (TIMES)}$$

Big-step semantics are typically closer to how an evaluator in an interpreter would be implemented, where an interpreter for a programming language should produce a value v when given a program e in context Γ iff $\Gamma \vdash e \Downarrow v$ is derivable using the language's big-step operational semantics.

To construct an evaluator from big-step operational semantics, each derivation rule can be implemented.

For example, consider the evaluator for arithmetic SIMPL expressions in OCaml with grammar

$$a \in \langle \text{ArithExpr} \rangle ::= z \mid a_1 + a_2 \mid a_1 - a_2 \mid a_1 * a_2 \mid a_1 / a_2$$

```

1  exception Stuck;;
2
3  (* omitted: definition of type store *)
4
5  (* ADT for the AST *)
6  type arith_expr = Plus of arith_expr * arith_expr
7                  | Minus of arith_expr * arith_expr
8                  | Times of arith_expr * arith_expr
9                  | Div of arith_expr * arith_expr
10                 | Int of int
11
12  let rec evaluator (s : store) (e : arith_expr) =
13    match e with
14    | Plus (e1, e2) →
15      (* recursive call for left expression *)
16      (match (evaluator s e1) with
17       (Int z1, s1) →
18         (* recursive call for right expression *)
19         match (evaluator s1 e2) with
20          (* add the expressions *)
21          (Int z2, s2) → (Int (z1 + z2), s2)
22          | _ → raise Stuck
23       | _ → raise Stuck)
24    | Minus (e1, e2) →
25      (* recursive call for left expression *)
26      (match (evaluator s e1) with
27       (Int z1, s1) →
28         (* recursive call for right expression *)
29         match (evaluator s1 e2) with
30          (* subtract the expressions *)
31          (Int z2, s2) → (Int (z1 - z2), s2)
32          | _ → raise Stuck
33       | _ → raise Stuck)
34    | Times (e1, e2) →
35      (* recursive call for left expression *)
36      (match (evaluator s e1) with
37       (Int z1, s1) →
38         (* recursive call for right expression *)
39         match (evaluator s1 e2) with
40          (* multiply the expressions *)
41          (Int z2, s2) → (Int (z1 * z2), s2)
42          | _ → raise Stuck
43       | _ → raise Stuck)
44    | Div (e1, e2) →
45      (* recursive call for left expression *)
46      (match (evaluator s e1) with
47       (Int z1, s1) →
48         (* recursive call for right expression *)
49         match (evaluator s1 e2) with
50          (* divide the expressions *)
51          (Int z2, s2) → (Int (z1 / z2), s2)
52          | _ → raise Stuck
53       | _ → raise Stuck)
54    | Int z → (Int z, s)

```

3.6 Syntactic Sugar

Rather than giving semantics directly for all expressions in a language, we often define a smaller core language, then give semantics for this core language, and define the expressions of the full language in terms of the core language. This helps to simplify the implementation of the corresponding interpreter as there are fewer semantic rules. For example, we can reduce the set of operators of SIMPL as follows:

$$\langle op \rangle ::= + \mid * \mid /$$

Then we define the expressions of the language that are not in the core language as *syntactic sugar*¹ for the core language expressions. Since we remove only $-$, consider

$$e_1 - e_2 \equiv e_1 + (-1 * e_2)$$

To implement an interpreter, we can then implement an evaluator based on the core SIMPL semantics. To give a SIMPL AST to this evaluator, we need to *desugar*, i.e. rewrite syntactic sugar into core language constructs, the SIMPL program by replacing each instance of syntactic sugar with its corresponding core SIMPL expression. The evaluator for SIMPL is then implemented in terms of the desugarer and core language evaluator, i.e.

$$\text{Evaluator}_{\text{SIMPL-AST}} = \text{Evaluator}_{\text{coreSIMPL-AST}} \circ \text{Desugarer}_{\text{SIMPL-AST}}$$

A desugarer performs a language-to-language transformation and is therefore simply a kind of compiler from the full language to the core language.

Define SIMPL^+ , which adds additional expressions and Boolean operators to the syntax of SIMPL, i.e.

$$e \in \langle expr \rangle ::= \dots \mid \mathbf{not} \ e$$

$$\odot \in \langle op \rangle ::= \dots \mid \mathbf{and} \mid \mathbf{or}$$

Then, rather than defining new rules in the operational semantics for these operators, we can add syntactic sugar to SIMPL to provide their semantics, e.g.

$$\mathbf{not} \ e \equiv \mathbf{if} \ e \ \mathbf{then} \ \mathbf{false} \ \mathbf{else} \ \mathbf{true}$$

$$e_1 \ \mathbf{or} \ e_2 \equiv \mathbf{if} \ e_1 \ \mathbf{then} \ \mathbf{true} \ \mathbf{else} \ e_2$$

$$e_1 \ \mathbf{and} \ e_2 \equiv \mathbf{if} \ e_1 \ \mathbf{then} \ e_2 \ \mathbf{else} \ \mathbf{false}$$

While we can write programs directly in the core language, SIMPL programs with the same functionality are easier to read and write, and SIMPL^+ programs are even easier. Therefore, we can view syntax as a user interface that is implemented using the underlying semantics and syntactic sugar definitions.

¹a nicer or more convenient way to write something that can be translated into more basic language constructs.